Cahier des charges

Scraper Google My Business (scan automatique) & Bot de contact par formulaires

Spécification complète pour reconstruire à l'identique les deux outils sur un autre Mac : fonctionnalités, règles métier, paramètres, structures de données, interface, garde-fous et procédure d'installation.

Version du 28/10/2026 — état réel de l'outil en production (version France), avec en annexe la variante États-Unis.

Sommaire

1. Objet et périmètre
2. Architecture générale
3. Environnement technique et installation
4. Configuration Google Sheets
5. Module 1 — Dashboard de scraping manuel
6. Module 2 — Scan automatique continu
7. Module 3 — Bot de contact
8. Structure du Google Sheet
9. Bases de données locales
10. Exploitation et supervision
11. Règles et garde-fous
12. Limites connues
13. Recette (tests d'acceptation)
14. Annexe A — Modèles de message
15. Annexe B — Variante États-Unis

1. Objet et périmètre

L'outil sert à constituer une base de fiches Google Maps (Google Business Profile) d'entreprises françaises **mal notées** et disposant d'un **site web**, puis à contacter ces entreprises via le **formulaire de contact de leur site** avec des messages prédéfinis (offre Review Cleaner : suppression d'avis négatifs).

Il se compose de **deux applications séparées** (exigence : le bot de contact ne doit pas être intégré au scraper) :

- **Application « Scraper »** (port 8600) : un dashboard de recherche manuelle et une page « Scan automatique » qui tourne en continu sur toutes les catégories × toutes les communes de France.
- **Application « Bot de contact »** (port 8601) : prépare les formulaires, les fait valider, les envoie, et reporte les statuts dans le Google Sheet.

Les données transitent par un **Google Sheet** unique (onglet « Scan auto ») qui sert à la fois de sortie du scan et d'entrée du bot de contact (colonne Opt-in).

2. Architecture générale

Élément	Rôle	Fichier(s)
Dashboard manuel	Recherche ponctuelle par catégories + villes/départements + filtres, export Sheets/CSV	`app.py`
Page Scan automatique	Pilotage, réglages et suivi en direct du scan continu	`pages/1_🤖 _Scan_automatique.py`
Moteur de scan	Processus détachés (1 par onglet), file de tâches, enregistrement, synchro Sheets	`crawler.py`
Scraper Google Maps	Pilotage d'un Chromium (Playwright) sur google.com/maps	`scraper.py`
Exclusions	Établissements publics et banques (+ mots-clés libres)	`exclusions.py`
Dédoublonnage par site	Clé « site_key » (1 fiche par site web)	`sites.py`
Catégories / communes	4 053 catégories GBP (FR/EN), 35 000 communes (geo.api.gouv.fr)	`categories.py`, `geo.py`, `data/`
Interface bot de contact	Profil, modèles, envoi groupé, à compléter, à vérifier, analytics, historique	`contact/app.py`
Moteur bot de contact	Lecture Sheet, préparation et envoi en parallèle, écriture des statuts	`contact_bot.py`
Remplissage de formulaires	Trouver le formulaire, classer les champs, remplir, valider, soumettre	`formfill.py`
Ouverture manuelle	Ouvre un formulaire prérempli dans un Chrome visible (champs manquants en rouge)	`ouvrir_formulaire.py`

> **Principe clé :** tout l'état (fiches, progression, réglages, commandes Démarrer/Pause/Arrêter) est stocké dans des bases SQLite locales (`data/scan.db` , `data/contact.db`). Les interfaces ne font qu'écrire des commandes et lire l'état ; les moteurs tournent en **processus détachés** (`nohup + caffeinate -is`) et survivent à la fermeture ou au redémarrage des interfaces. Tout reprend exactement où ça s'était arrêté.

3. Environnement technique et installation

- macOS, Python 3.11, environnement virtuel `.venv` dans le dossier du projet.
- Dépendances (`requirements.txt`) : `streamlit>=1.38` , `playwright>=1.47` , `pandas>=2.2` , `requests>=2.32` , `gspread>=6.1` , `google-auth>=2.34` .
- Navigateur : `python -m playwright install chromium` .
- Lancement : `streamlit run app.py` (port 8600, fichier `.streamlit/config.toml`) et `streamlit run contact/app.py --server.port 8601` . Deux raccourcis `.command` double-cliquables.
- Le dossier ne doit pas être dans iCloud Drive (bases SQLite en écriture continue).

```
python3.11 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python -m playwright install chromium
.venv/bin/streamlit run app.py                                  # scraper (8600)
.venv/bin/streamlit run contact/app.py --server.port 8601       # bot de contact (8601)
```

4. Configuration Google Sheets

1. Créer un projet Google Cloud, activer **Google Sheets API** et **Google Drive API**.
2. Créer un **compte de service**, générer une clé JSON, l'enregistrer en `credentials.json` à la racine du projet.
3. Partager le Google Sheet **en Éditeur** avec l'adresse email du compte de service (affichée dans le dashboard).
4. Coller le lien du Sheet dans le dashboard (barre latérale « Google Sheets » → Enregistrer). Le lien et le chemin de la clé sont stockés dans `.settings.json`.

Écritures toujours en mode `RAW` (sinon « +33 6… » est interprété comme une formule). Le Sheet est en locale fr_FR : les formules utilisent « ; » comme séparateur.

> La clé `credentials.json` donne accès à tous les Sheets partagés avec ce compte de service : ne pas la diffuser. Un autre Mac peut utiliser sa propre clé (nouveau compte de service) pour rester cloisonné.

5. Module 1 — Dashboard de scraping manuel (port 8600, page d'accueil)

- **Catégories** : sélection parmi les 4 053 catégories Google en français, filtrables par famille (25 familles). Recherche « large » par mot-clé (ex. « restaurant ») : bouton « Ajouter ces N catégories » qui ajoute toutes les catégories correspondantes (recherche sans accents sur libellés FR et EN).
- **Zone** : saisie dynamique de communes (autocomplétion sur ~35 000 communes) *ou* un/plusieurs départements saisis par numéro (ex. « 02 »), avec mode de couverture du département. Filtre post-recherche : l'adresse doit être dans la zone.
- **Filtres** : note min–max (plage), nombre d'avis minimum, site web oui/non/indifférent, case « Exclure établissements publics et banques ».
- **Réglages** : navigateur invisible, délai entre fiches (0,5–6 s), max fiches par recherche (10–120).
- **Résultats** : tableau en direct, dédoublonnage par site web, export Google Sheets (nouvel onglet daté) ou CSV.
- Colonnes : Nom, Catégorie, Note, Nb avis, Adresse, Téléphone, Site web, Lien Google Maps, Recherche.

6. Module 2 — Scan automatique continu

6.1 Objectif

Parcourir sans s'arrêter **toutes les catégories × toutes les communes de France** (objectif réglable, 50 000 puis 69 000 fiches) et ne garder que les fiches correspondant aux critères, envoyées automatiquement dans l'onglet « Scan auto ».

6.2 Critères de rétention (réglages actuels)

Paramètre	Valeur	Remarque
Note min / max	0,8 / 3,8	valeur initiale 0,8–3,5 ; fiches sans note exclues
Avis minimum	1	
Site web	obligatoire	
Exclure publics et banques	oui	voir 6.6
Exclusions supplémentaires	medecin, docteur, médecin, clinique	mots-clés libres (nom + catégorie)
Objectif	69 000	« S'arrêter à l'objectif » désactivé
Max résultats / recherche	120	plafond Google Maps
Délai	1,0 s	multiplié par un aléa 0,7–1,3 ; pause 1–3 × délai entre recherches
Onglets en parallèle	4	1 processus = 1 navigateur
Envoi vers Sheets	toutes les 3 min	par l'onglet n° 1

6.3 Recherche Google Maps (scraper.py)

- Chromium Playwright avec **user-agent Chrome normal** (sinon Google sert une vue sans nombre d'avis), locale fr-FR, fenêtre 1300×900. URL : `https://www.google.com/maps/search/<requête>?hl=fr`. Page de consentement acceptée/refusée automatiquement.
- Requête = `« <catégorie> <commune> <code postal> »` (ex. « Plombier Lyon 69003 »).
- Défilement de la liste `div[role=feed]` jusqu'à 120 cartes, fin de liste ou 4 défilements sans nouveauté.
- **Pré-filtrage sur la carte** (sans ouvrir la fiche) : note lue sur l'étoile, nombre d'avis, présence du bouton « Site Web ». Les cartes hors critères sont écartées immédiatement (gain de temps majeur).
- Fiche ouverte dans un second onglet : nom (h1), catégorie, adresse, téléphone, site (`a[data-item-id=authority]`), note et avis.
- Identifiant unique : place_id extrait du lien (`!1s(0x…:0x…)`). Une fiche déjà vue n'est jamais rouverte.
- Détection de blocage : URL `/sorry/` → erreur « captcha ».

6.4 File de travail intelligente

- **8 vagues** combinant type de catégorie et taille de commune : G = ~227 catégories « générales » (`data/generic.txt` : restaurants, artisans du BTP, garages, beauté, santé, immobilier…), S = toutes les autres. Ordre : G ≥ 20 000 hab. → G 5 000–20 000 → S ≥ 20 000 → G 1 000–5 000 → S 5 000–20 000 → G < 1 000 → S 1 000–5 000 → S < 1 000.
- Zones = une par (commune, code postal), triées de la plus peuplée à la moins peuplée (cache `data/zones.json`).
- Chaque catégorie avance sur ses zones via un curseur. À chaque tâche, la catégorie choisie est celle dont le **rendement récent** (moyenne mobile exponentielle des fiches retenues, poids 0,3) est le plus élevé ; les catégories jamais testées passent en priorité, avec tirage aléatoire à égalité.
- Attribution **atomique** des tâches entre onglets (transaction SQLite `begin immediate`) : deux onglets ne font jamais la même recherche. Tâche échouée → table `retry` (réessayée plus tard).
- Au démarrage, purge des curseurs des catégories devenues exclues.

6.5 Dédoublonnage

- Une seule fiche **par site web** : clé `site_key` = domaine enregistrable (ex. `wash-totalenergies.fr`), ou lien complet sans paramètres pour les plateformes partagées (Doctolib, Facebook, Instagram, PagesJaunes, Wix, Booking, free.fr, WordPress.com…). Index unique SQLite sur `site_key` + insertion « or ignore » entre onglets.
- Une fiche déjà vue (retenue ou non) n'est jamais réexaminée (table `seen`).

6.6 Exclusions (établissements publics et banques)

- **Catégorie Google** : toute la famille « Administrations & services publics » (sauf commissaire/huissier de justice, conseil), plus motifs : administration, gouvernement, services publics, police, douane, poste, municipal, banque, bank, bancaire, distributeur de billets, institution financière, caisse d'épargne…
- **Nom** : impôts, finances publiques, trésor public, URSSAF, CAF, CPAM, Assurance maladie, Ameli, MSA, CARSAT, Pôle emploi / France Travail, mission locale, ambassade, consulat, bureau de poste, conseil départemental/régional, communauté de communes, office de tourisme, CROUS, enseignes bancaires (Crédit Agricole, BNP, LCL, Société Générale, CIC, HSBC, Boursorama…). Mairie/préfecture/tribunal/gendarmerie *seulement en début de nom* (« Parking Mairie » reste exclu, « Café de la Préfecture » non).
- « Banque » dans le nom n'exclut que si la catégorie est vide ou financière (« Bistro de la Banque » est gardé).
- **Domaine du site** : `.gouv.fr`, service-public, urssaf, caf, ameli, msa, laposte, banques…
- Les catégories exclues ne sont même pas recherchées (temps gagné).

6.7 Synchronisation Google Sheets

- Toutes les N minutes, l'onglet n° 1 ajoute (`append_rows`, RAW) les fiches non encore exportées, jusqu'à 5 000 par passe, puis les marque exportées. Crée l'onglet « Scan auto » avec en-têtes s'il n'existe pas.
- Jamais de réécriture des lignes existantes : le scan ne renvoie pas ce qu'il a déjà envoyé.
- Variable d'environnement `GMB_SCAN_DB` (base de test) ⇒ aucune écriture dans le Sheet.

6.8 Robustesse

- Blocage Google (captcha) : **pause automatique de tous les onglets** 30 min, doublée à chaque nouveau blocage (max 240 min) ; bouton « 🔓 Reprendre maintenant ».
- 3 erreurs consécutives sur un onglet ⇒ pause 5 min. Navigateur relancé toutes les 150 recherches.
- Interrupteur « 👁️ Navigateur visible » : appliqué au prochain redémarrage du navigateur.
- Processus lancés par `/bin/sh -c "nohup caffeinate -is … &"` (indépendants de l'interface, Mac maintenu éveillé).
- Base SQLite en mode WAL, ouverture avec réessais (« database is locked ») ; vérification de vie par PID + battement de cœur.

6.9 Interface « 🤖 Scan automatique »

- Boutons ▶️ Démarrer (lance N onglets détachés), ⏸️ Pause, ⏹️ Arrêter, interrupteur navigateur visible.
- Formulaire de réglages (§ 6.2), sélection du nombre d'onglets (1–4).
- Suivi rafraîchi automatiquement : état de chaque onglet (vague, recherche en cours, dernière fiche), métriques (fiches retenues, objectif, rythme réel), graphique, meilleures catégories, dernières fiches, export CSV, journal.

7. Module 3 — Bot de contact (application séparée, port 8601)

7.1 Principe et cycle de vie d'une fiche

Le bot lit le Sheet (onglet « Scan auto »), prend les lignes **Opt-in = « oui »** avec un site web et une colonne « Statut contact » vide, à partir de la ligne « Commencer à la ligne » (réglage, actuellement 2). Il **prépare** chaque formulaire (trouvé et rempli, **sans envoi**), puis n'envoie que ce que l'utilisateur a validé.

Statut	Signification
à valider	formulaire trouvé et rempli, en attente de validation (envoi groupé ou un par un)
à valider + champs manquants	un champ obligatoire n'a pas pu être rempli → onglet « ⚠️ À compléter »
à envoyer / envoi en cours	validé par l'utilisateur, dans la file d'envoi
envoyé	confirmation visible (message de remerciement, changement de page ou formulaire disparu)
à vérifier	envoyé mais aucune confirmation visible
échec envoi	refus du site (anti-robot, erreur), formulaire introuvable, site trop lent après 2 essais
pas de formulaire / captcha / site inaccessible / erreur	préparation impossible
passé / ne jamais contacter	choix de l'utilisateur
doublon (même site) / exclu	écrit directement dans le Sheet pour les lignes non traitées

7.2 Parallélisme et fiabilité

- **8 onglets** en parallèle (réglable). Chaque site est traité dans un **processus séparé** (`python contact_bot.py --site`) **coupé de force après 45 s** (le site passe en « site inaccessible — bloqué plus de 45 s ») : un site bloquant ne peut plus geler un onglet.
- Les envois validés passent **avant** la préparation. Réservation atomique d'un envoi par onglet (aucun double envoi).
- Site trop lent avant soumission ⇒ **un seul** nouvel essai plus tard ; jamais de renvoi après une soumission.
- Plafond « formulaires préparés à l'avance » (1 500) : au-delà, la préparation se met en pause jusqu'aux prochains envois.
- Limite d'envois par jour (tentatives) modifiable à tout moment, sans maximum, bouton + par pas de 5 000.
- Relecture du Sheet toutes les 2 min si la file est vide, sinon toutes les 10 min ; écriture des statuts toutes les 2 min.
- Exclusions identiques au scan (+ mots-clés supplémentaires du scan) ; doublons de site ⇒ marqués « doublon (même site) ».

7.3 Recherche du formulaire (formfill.py)

1. Ouvrir le site (25 s max), accepter le bandeau cookies, faire défiler (formulaires chargés tardivement, ex. Wix).
2. Chercher un formulaire contenant une zone de message (`textarea`), y compris dans les iframes.

3. Sinon suivre jusqu'à 3 liens « Contact / Contactez-nous / Nous écrire / Devis / Formulaire » du même site (hors mentions légales, CGV, recrutement), sinon essayer `/contact`, `/contactez-nous`, `/nous-contacter`, `/contact.html`, `/contact.php`, `/fr/contact`.

7.4 Remplissage

- Classement de chaque champ visible par libellé, placeholder, nom, id, attribut autocomplete : email, téléphone, nom complet, prénom, nom, société, objet, code postal, pays, ville, adresse, site web, message. Un champ « Nom » seul (sans prénom) reçoit le nom complet. Champ numérique ⇒ chiffres uniquement.
- **Champs cachés jamais remplis** (pièges anti-robots / honeypots).
- Cases à cocher : consentement RGPD / politique de confidentialité cochées ; **jamais** newsletter, offres commerciales, partenaires, « être recontacté par téléphone/SMS ». Cases stylisées gérées (libellé visible).
- Listes déroulantes : option « Autre / Information / Demande / Contact… » de préférence ; liste de pays ⇒ pays du profil.
- Après remplissage, validation du navigateur (`checkValidity`) : listes et boutons radio obligatoires restés vides remplis avec la première option valable ; tout champ encore refusé ⇒ « À compléter » (le formulaire ne part pas).
- Message coupé par une limite de caractères du site ⇒ alerte « raccourcis-le », décoché par défaut dans l'envoi groupé.
- Valeurs saisies à la main pour un libellé donné (onglet « À compléter ») réutilisées à l'envoi.

7.5 Captchas, cookies, soumission

- Captchas visibles (reCAPTCHA case, hCaptcha, Turnstile, image, calcul) ⇒ statut « captcha ». **Aucun contournement.** Le reCAPTCHA v3 invisible n'est pas compté comme captcha.
- Bandeaux cookies acceptés automatiquement (OneTrust, Didomi, Axeptio, Cookiebot, Complianz, CookieYes, tarteaucitron, iubenda, Quantcast… + boutons « Tout accepter / Accepter / J'accepte / OK »), y compris juste avant l'envoi.
- Soumission : bouton submit (clic direct si recouvert), sinon `requestSubmit()`. Attente 5 s puis lecture : texte de refus (« reCAPTCHA a échoué », « utilisation abusive », « une erreur s'est produite », « formulaire non valide »…) ⇒ échec envoi ; texte de succès (« merci », « bien envoyé », « thank you »…) ⇒ envoyé ; changement d'URL ou formulaire disparu ⇒ envoyé ; sinon ⇒ à vérifier. Captures avant et après envoi conservées.

7.6 Interface du bot de contact

- **Profil** : prénom, nom, société, email, téléphone, site, adresse, code postal, ville, pays (France par défaut).
- **4 modèles de message** (objet + message) utilisés à tour de rôle (le moins utilisé d'abord). Variables : `{entreprise} {ville} {categorie} {note} {nb_avis} {site} {prenom} {nom} {societe} {email} {telephone} {mon_site}`. Contrôle : la société doit figurer dans le message.
- Réglages : formulaires préparés à l'avance, onglets en parallèle, ligne de départ, limite par jour, navigateur visible.
- Boutons ▶️ Démarrer / ⏸️ Pause / ⏹️ Arrêter ; barre d'état rafraîchie toutes les 15 s (activité de chaque onglet, compteurs).
- 📦 **Envoi groupé** : liste (1 000 par défaut, sans maximum) des formulaires prêts sans champ manquant, cases cochées sauf alertes, bouton « 📤 Envoyer les N formulaires sélectionnés ».

- ⚠️ **À compléter** : un champ de saisie par champ manquant, « 🖥️ Ouvrir dans Chrome » (formulaire prérempli, champs en rouge, l'utilisateur envoie lui-même), « ✔ Je l'ai envoyé », « ✅ Compléter et envoyer », Passer, Ne jamais contacter.
- 🔎 **À vérifier** : capture après envoi, « ✔ Bien arrivé », « 🖥️ Ouvrir dans Chrome », « ❌ Pas arrivé ».
- ✅ **À valider (un par un)** : capture, champs remplis, objet et message modifiables, Envoyer / Passer / Ne jamais contacter.
- 📊 **Analytics** (lu dans le Sheet) : par modèle, envoyés, réponses, taux de réponse, intéressés, taux d'intérêt, STOP.
- 📜 **Historique** : 300 dernières actions avec captures.

8. Structure du Google Sheet

Colonne (onglet « Scan auto »)	Écrite par
Nom, Catégorie, Note, Nb avis, Adresse, Téléphone, Site web, Lien Google Maps, Recherche, Date	Scan
Optin (« oui »)	Utilisateur (consentement géré dans Mailchimp)
Statut contact, Date contact, Modèle	Bot de contact (ligne retrouvée par le lien Google Maps)
Réponse (liste : intéressé / pas intéressé / STOP)	Utilisateur

Les colonnes du bot sont créées si absentes ; les en-têtes sont comparés sans accents ni espaces (« Optin » = « Opt-in »). Onglet **« Analytics »** : une ligne par modèle + TOTAL, formules NB.SI.ENS (envoyés = envoyé + à vérifier ; réponses ; intéressés ; STOP) et pourcentages, mis à jour automatiquement.

9. Bases de données locales (SQLite, mode WAL)

```
scan.db
  places(place_id PK, nom, categorie, note, nb_avis, adresse, telephone, site_web, lien_maps,
recherche,
         scanned_at, exported, site_key UNIQUE)
  seen(place_id PK)                      -- fiches déjà examinées
  cat_stats(categorie PK, searches, kept, new, ema_kept, ema_new)
  cursors(tier, categorie, zone_idx)     -- progression par vague et catégorie
  retry(tier, categorie, zone_idx, attempts)
  searches(id, ts, query, tier, cards, new, kept, secs, error)
  state(key PK, value JSON)              -- config, commandes, état des onglets, blocage

contact.db
  contacts(site_key PK, maps_link, nom, site, ville, categorie, note, nb_avis, status,
template, objet, message,
           form_url, shot, shot_after, detail, filled, missing, extra, prepared_at, sent_at,
sheet_synced)
  state(key PK, value JSON)              -- config (profil, modèles, réglages), commande, pid,
battement
```

10. Exploitation et supervision

- Journaux : `data/scan.log`, `data/contact.log`, `data/contact.stdout.log` (les messages macOS « MallocStackLogging » sont sans effet).
- Contrôle régulier (fait toutes les 20 min pendant l'exploitation) : bot vivant, erreurs, répartition des statuts, écriture des statuts dans le Sheet, état du scan.
- Après modification du code : redémarrer l'interface concernée ; pour le bot de contact, Arrêter, attendre la fin du processus, puis Démarrer. Les sites étant traités chacun dans un nouveau processus, les corrections de `formfill.py` s'appliquent au site suivant sans redémarrage.
- Repères de production (version France, 2 jours) : ~31 000 fiches scannées ; ~29 000 lignes traitées par le bot ; ~5 300 envois confirmés ; environ 1 site sur 3 possède un formulaire exploitable ; ~25–30 % des envois sans confirmation visible.

11. Règles et garde-fous (à conserver)

> - **Aucun envoi sans action de l'utilisateur** : envoi groupé sur une liste cochée, ou un par un. Pas d'envoi automatique.
> - **Aucun contournement de captcha** ni de protection anti-robot ; les champs cachés (honeypots) ne sont jamais remplis.
> - Jamais de case newsletter / offres commerciales cochée ; seule la case de consentement nécessaire à l'envoi.
> - Une seule prise de contact par site web ; fiches marquées « ne jamais contacter » définitivement écartées.
> - Seules les lignes Opt-in = « oui » sont contactées ; la conformité RGPD (consentement, droit d'opposition, réponses STOP) relève de l'utilisateur.
> - Le scraping de Google Maps est contraire aux conditions d'utilisation de Google : volumes raisonnables, délais, pause automatique en cas de blocage.

12. Limites connues

- Rendement du scan décroissant avec le temps (catégories de niche, fiches déjà vues).
- Des organismes publics peuvent passer le filtre (ex. syndicat intercommunal, établissement public de santé, Cap emploi) : compléter les exclusions au besoin.
- Certains hébergeurs bloquent l'IP après de nombreuses visites (« site inaccessible » en série) : non contourné.
- Les sites protégés par reCAPTCHA v3 refusent souvent l'envoi automatisé (« échec envoi ») : à envoyer à la main.
- Une modification de l'interface de Google Maps peut nécessiter d'adapter les sélecteurs de `scraper.py`.

13. Recette (tests d'acceptation)

1. Une recherche « Dépanneur auto Paris 75011 » renvoie des fiches avec note, avis, site, téléphone, adresse remplis.
2. Scan sur base de test (`GMB_SCAN_DB`) : 4 onglets sans recherche en double, aucune fiche hors critères ni doublon de site.
3. Exclusions : « Mairie de Lyon », « URSSAF », « Crédit Agricole » exclus ; « Bistro de la Banque », « Café de la Préfecture » gardés.
4. Arrêt puis redémarrage du scan : reprise sans refaire les recherches déjà faites ; pas de doublon dans le Sheet.
5. Bot de contact sur un formulaire de démonstration (`httpbin.org/forms/post`) : préparation « à valider », envoi « envoyé ».
6. Formulaire avec liste obligatoire, champ numérique et case RGPD : rempli correctement, newsletter non cochée.
7. Site qui ne répond pas : coupé à 45 s, onglet libéré.
8. Statuts reportés dans le Sheet en moins de 3 minutes ; Analytics à jour.
9. Arrêt brutal pendant un envoi : au redémarrage, la fiche repasse « à envoyer » (pas de perte ni de double envoi).

Annexe A — Modèles de message (version française)

Modèle 1 — Objet : « Votre note Google My business »

```
Bonjour,

Nous avons consulté votre fiche Google et remarqué plusieurs avis négatifs qui impactent votre
réputation en ligne.

Review Cleaner est spécialisé dans la suppression d'avis négatifs sur Google. Nous pouvons
prendre en charge ces avis et
effectuer l'ensemble des démarches nécessaires à leur suppression.

Est-ce que cela pourrait vous intéresser ?

Merci,
L'équipe Review Cleaner
```

Modèle 2 — Objet : « Nous pouvons supprimer vos avis négatifs Google »

```
Bonjour,

En consultant votre établissement sur Google, nous avons constaté que plusieurs avis négatifs
affectent votre note et restent
visibles auprès de vos futurs clients.

Review Cleaner peut prendre en charge leur suppression et les démarches associées.

Souhaitez-vous que nous intervenions sur ces avis ?

Merci,
L'équipe Review Cleaner
www.reviewcleaner.com
```

Modèle 3 — Objet : « Vos avis Google font fuir des clients »

```
Bonjour,

Nous avons identifié plusieurs avis négatifs sur votre fiche Google.

Notre métier chez Review Cleaner est justement d'aider les entreprises à les faire supprimer et
à protéger leur réputation en ligne.

Nous pouvons nous occuper directement de votre situation. Est-ce que cela pourrait vous
intéresser ?

Merci,
L'équipe Review Cleaner
```

Modèle 4 — Objet : « Nous avons identifié des avis négatifs sur votre fiche »

```
Bonjour,

Nous avons consulté votre fiche Google et plusieurs avis négatifs pénalisent actuellement votre
note et votre image auprès de
vos futurs clients.

Review Cleaner est spécialisé dans la suppression d'avis négatifs sur Google. Nous pouvons
prendre en charge l'ensemble des
démarches nécessaires pour votre établissement.

Vous trouverez toutes les informations sur notre site web. Nous pouvons démarrer la prise en
charge de votre dossier sous 24h.

Merci,
L'équipe Review Cleaner
www.reviewcleaner.com
```

Annexe B — Variante États-Unis (paquet « Scraper-US »)

- **Zones** : 20 813 villes des 50 États + DC (Census Bureau, estimations 2023 ; requête « Austin, TX ») + les 5 arrondissements de New York. Mêmes 8 vagues par population.
- **Catégories** : libellés anglais (champ « en »), ~195 catégories générales adaptées aux petites entreprises US.

- **Google Maps** : locale en-US, fuseau America/New_York, `?hl=en&gl=us`.
- **Exclusions** : administrations (city/town/county/state of, city hall, courthouse, DMV, post office/USPS, police, sheriff, fire department, public schools/school district, IRS, social security, armed forces, embassy…), banques et credit unions, domaines `.gov` `.mil` `.edu`, `k12`, `ci./co.*.us`, `cityof…` ; famille « Government & Public Services » sauf commerces (surplus, terrains de sport, photos d'identité…).
- **Grandes enseignes** : liste d'environ 250 chaînes (domaines + débuts de nom : Walmart, Target, Home Depot, McDonald's, Starbucks, State Farm, RE/MAX, Planet Fitness…) et **détection automatique** : un même site web vu sur 5 fiches ou plus = chaîne (non exportée ; si déjà exportée, marquée « exclu (chaîne) » dans le Sheet et ignorée par le bot).
- **Formulaires en anglais** : liens « Contact Us / Get a quote / Request a quote / Estimate / Inquiry », chemins `/contact-us`…, messages de succès et d'échec anglais, cases « I agree to the Privacy Policy » cochées, « receive text messages / subscribe / marketing » jamais cochées, captchas texte anglais.
- **Modèles** traduits en anglais (Template 1 à 4), note affichée avec un point décimal, téléphone au format international.
- **Accès à distance** : mot de passe (haché PBKDF2 dans `.settings.json`, modifiable sur l'accueil) sur les trois pages ; liens publics via tunnels Cloudflare gratuits (changent à chaque lancement, inscrits dans `LIENS.txt`).
- **Installation** : « 1 - Installer.command » (uv + Python 3.11, modules, Chromium, cloudflared), « 2 - Lancer.command » (interfaces 8600/8601 + liens, ouvre les deux pages), « 3 - Arrêter.command ».